



计算机组成与体系结构

COMPUTER ORGANIZATION AND ARCHITECTURE

课程复习

四川大学网络空间安全学院
School of Cyber Science and Engineering, SCU

2023年6月8日

版权声明

课件中所使用的图片、视频等资源版权归原作者所有。

课件原创内容采用 创作共用署名 - 非商业使用 - 相同方式共享
4.0国际版许可证(Creative Commons BY-NC-SA 4.0
International License) 授权使用。

Copyright©四川大学网络空间安全学院计算机组成与体系结构课程组, 2020-2023

All images and other materials are copyright to their perspective owners.

Original materials in this lecture are released under the Creative Commons BY-NC-SA 4.0 International License.

Copyright©Teaching Group for Computer Organization and Architecture, SCSE, SCU, 2020-2023



A horizontal banner image featuring traditional Chinese architectural elements, including dark tiled roofs with ornate, upturned eaves and decorative finials. The left side of the banner is overlaid with a semi-transparent red filter. The text '期末考试' (Final Exam) is written in white, stylized Chinese characters across the center of the banner.

期末考试

题型

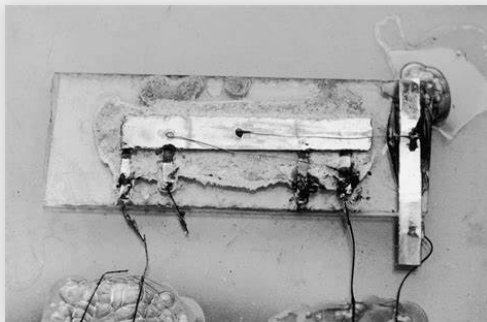
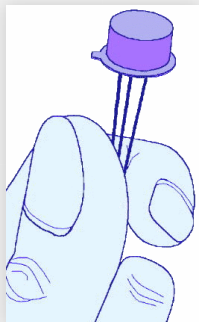
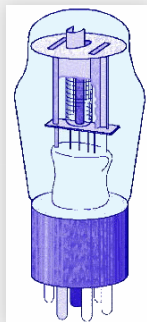
- 名词、缩写词解释 (7道)
- 单选题 (6道)
- 判断题 (6道)
- 填空题 (6道)
- 计算题 (4道)
- 分析题 (2道)



概述

计算机的发展历程

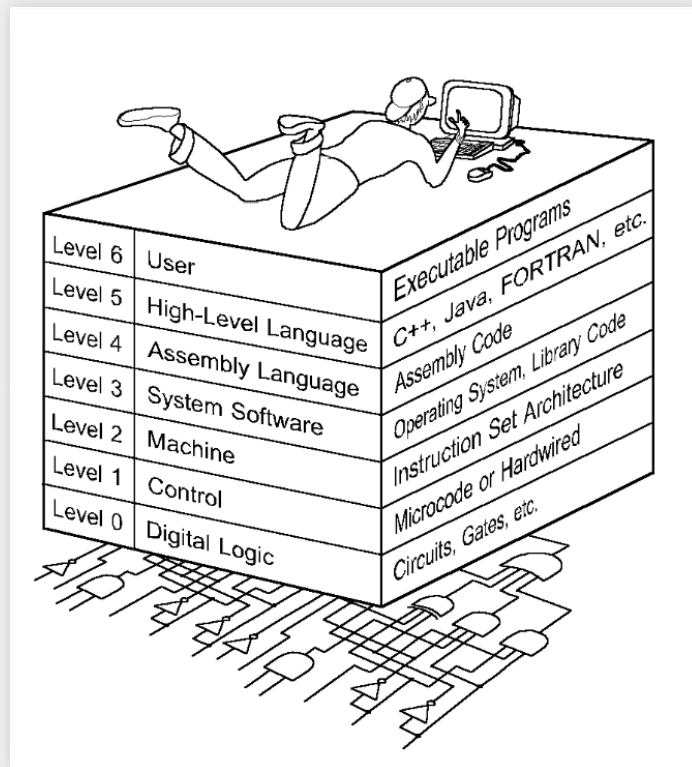
- 四代计算机处理器：
 - 电子管 (vacuum tube)
 - 晶体管 (transistor)
 - 集成电路(integrated circuits,IC)
 - 超大规模集成电路(very large scale integration,VLSI)
- 世界上第一台通用 (general-purpose) 计算机：ENIAC



来源: Linda Null & Julia Lober "Computer Organization & Architecture" (4th Edition), <https://www.wsj.com/articles/the-chip-that-changed-the-world-1535311622>, https://de.wikipedia.org/wiki/VLSI_Technology

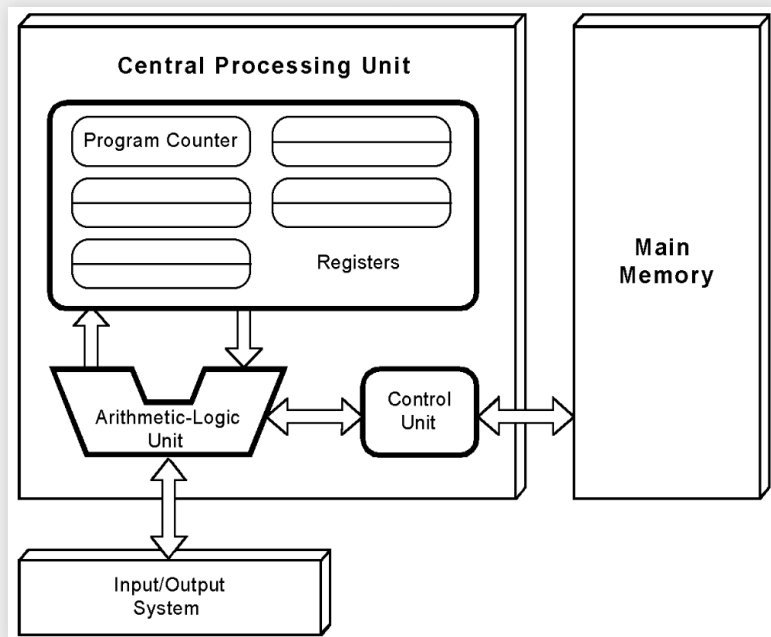
计算机层次结构

- 七层计算机层次结构
 - 每层的例子
- 云计算及其分类
 - 计算即服务、软件即服务、平台即服务、基础设施即服务等



冯诺依曼模型

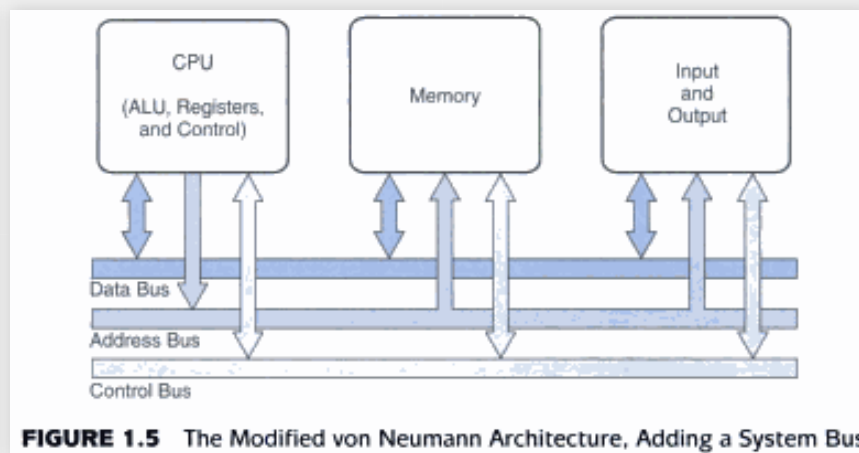
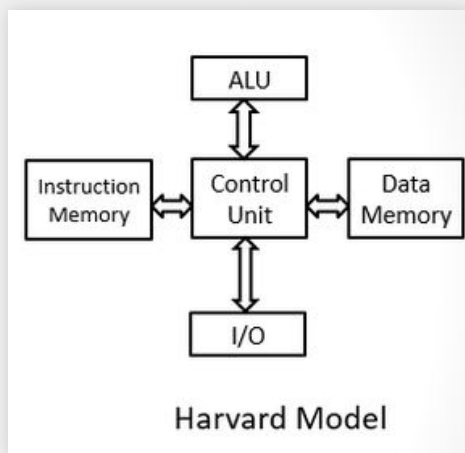
- 冯诺依曼计算机的组件：CPU、存储器、I/O
- 冯诺依曼周期：取-译码-执行
- 冯诺依曼瓶颈：指令和数据放在同一个存储器中，读取指令和传送数据采用同一个总线



来源: Linda Null & Julia Lobor "Computer Organization & Architecture" (4th Edition),

非冯诺依曼模型

- **哈佛架构**(Harvard architecture): 指令和数据分开存储, 采用两个总线分别传输指令和数据
- **系统总线模型** (System bus model): 采用数据总线、地址总线和控制总线



量化、单位、数据

- 量化：时间(s)，频率(Hz)，存储(bit/Byte)、比率(bps)
- 前缀乘数(Prefix multipliers):

前缀	二进制	十进制	前缀	二进制	十进制
Kilo- (K)	2^{10}	10^3	Milli- (m)	-	10^{-3}
Mega- (M)	2^{20}	10^6	Micro- (μ)	-	10^{-6}
Giga- (G)	2^{30}	10^9	Nano- (n)	-	10^{-9}
Tera- (T)	2^{40}	10^{12}	Pico- (p)	-	10^{-12}
Peta- (P)	2^{50}	10^{15}	Femto- (f)	-	10^{-15}
Exa- (E)	2^{60}	10^{18}	Atto- (a)	-	10^{-18}
Zetta- (Z)	2^{70}	10^{21}	Zepto- (z)	-	10^{-21}
Yotta- (Y)	2^{80}	10^{24}	Yocto- (y)	-	10^{-24}

小结

- 冯诺依曼体系结构
 - 冯诺依曼体系结构是什么？
 - 什么是冯诺依曼周期和瓶颈？
 - 什么是哈佛架构和系统总线模型？
- 量化&前缀乘数
 - 不同前缀对应的比特(bit)是什么？
 - 如何比较不同的前缀？
- 计算机的发展历程
 - 不同阶段的代表性技术是什么？
- 计算机体系的层次
 - 不同层的作用和例子是什么？

数字电路

按位计数法

- 按位记数法 (Positional numbering)
 - 二进制、十进制、十六进制
 - 不同进制间的转换

$$190_{10} = (21001)_3$$

$$\begin{array}{r} 3 \overline{) 190} \end{array}$$

$$\begin{array}{r} 3 \overline{) 63} \end{array}$$

$$\begin{array}{r} 3 \overline{) 21} \end{array}$$

$$\begin{array}{r} 3 \overline{) 7} \end{array}$$

$$\begin{array}{r} 3 \overline{) 2} \\ 0 \end{array}$$

$$0.8125_{10} = 0.1101_2$$

$$\begin{array}{r} .8125 \\ \times 2 \\ \hline 1.6250 \end{array}$$

$$\begin{array}{r} .6250 \\ \times 2 \\ \hline 1.2500 \end{array}$$

$$\begin{array}{r} .2500 \\ \times 2 \\ \hline 0.5000 \end{array}$$

$$\begin{array}{r} .5000 \\ \times 2 \\ \hline 1.0000 \end{array}$$

整数

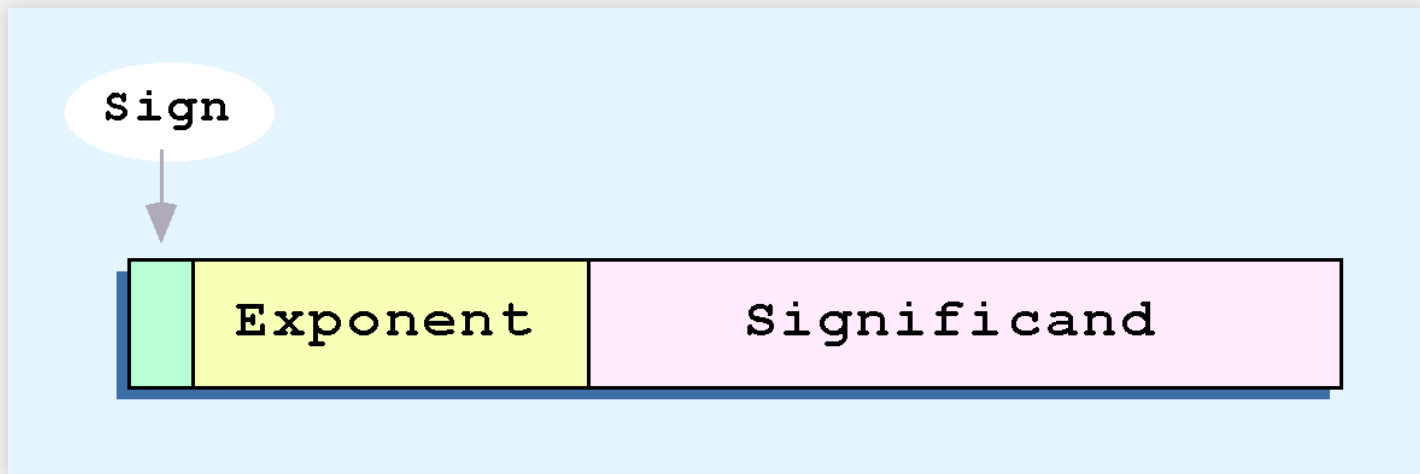
无符号整数(Unsigned integers): 采用对应的二进制数字表示

带符号整数 (Signed integers)

- 原码 (Signed magnitude) : 最高位代表符号位: 0 - 非负数, 1 - 负数
 - 例子: $-6 = (1110)_2$
- 一的补码/反码 (One' s complement)
 - 例子: $-6 = (1001)_2$
- 二的补码/补码 (Two' s complement)
- 例子: $-6 = (1010)_2$

浮点数

- 科学计数法(Scientific notation):采用符号位、指数位和有效数字位表示数字
- 指数位 (Exponent) 采用移码(excess code)
- 有效数位 (Significand) 采用规格化 (Normalization)
 - IEEE 754: 隐含位1



CRC循环校验码

- 模2除法
- 模函数

$$\begin{array}{r} 1011011 \\ 1101 \overline{) 1111101000} \\ \underline{1101} \\ 001010 \\ \underline{1101} \\ 01111 \\ \underline{1101} \\ 001000 \\ \underline{1101} \\ 01010 \\ \underline{1101} \\ 0111 \end{array}$$

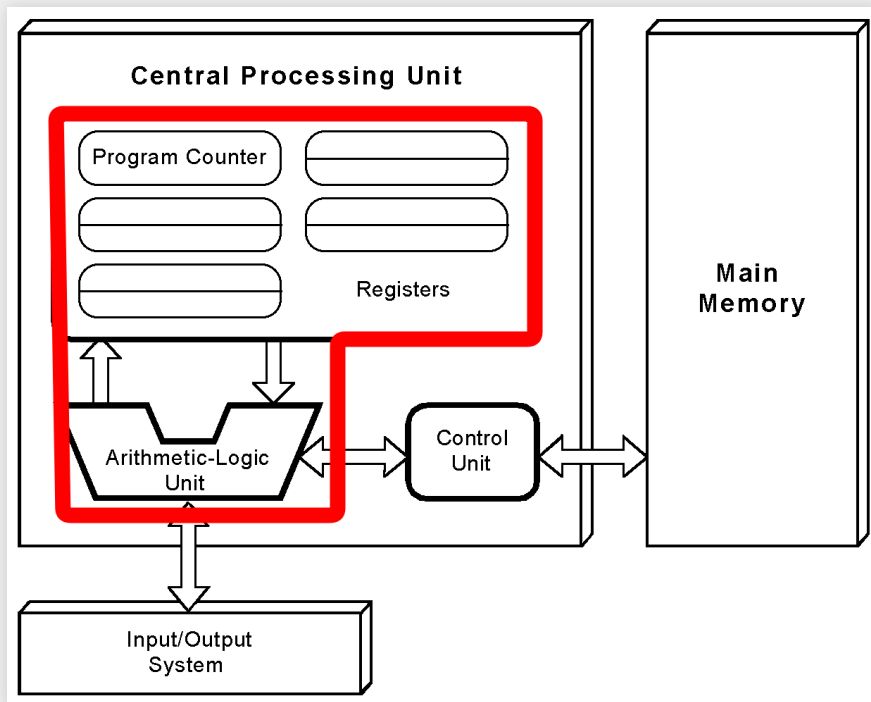
本章总结

- 数据表示
 - 无符号整数
 - 有符号整数
 - 浮点数
- CRC
 - 如何计算CRC校验码？
 - 如何验证CRC校验码？
- 数字电路

CPU基本知识和组织结构

CPU的组织结构

- 数据通路: 寄存器、算术逻辑单元、总线
- 控制单元



总线仲裁

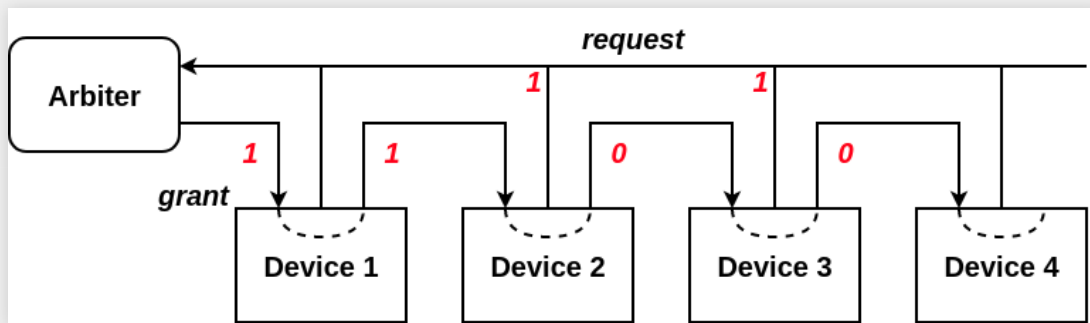
四种总线仲裁：它们是如何工作的？它们有什么特征？

- 菊链仲裁机制 (daisy chain arbitration)
- 集中式并行仲裁 (centralized parallel arbitration)
- 基于自检测的分布式仲裁 (distributed arbitration using self-detection)
- 基于冲突检测的分布式仲裁 (distributed arbitration using collision-detection)

菊链仲裁机制

菊链仲裁

- 所有设备共用一个控制线发送请求
- 控制许可按优先级从高到低传输
- 发送控制请求的设备中优先级最高的设备获得控制许可并清除控制许可

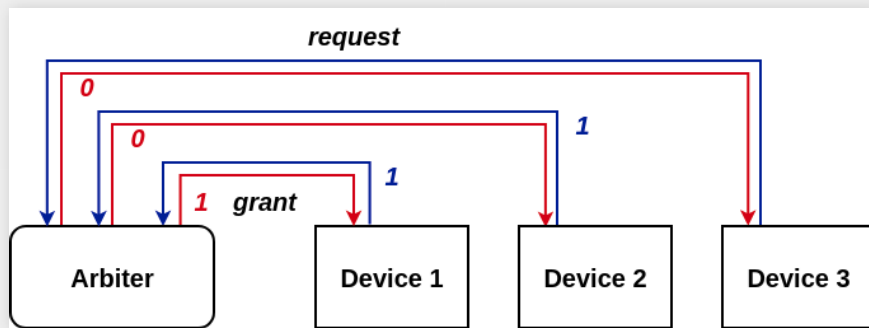


来源: <http://www.helpmykidlearn.ie/activities/5-7/detail/daisies-chains>

集中式并行仲裁

集中式并行仲裁

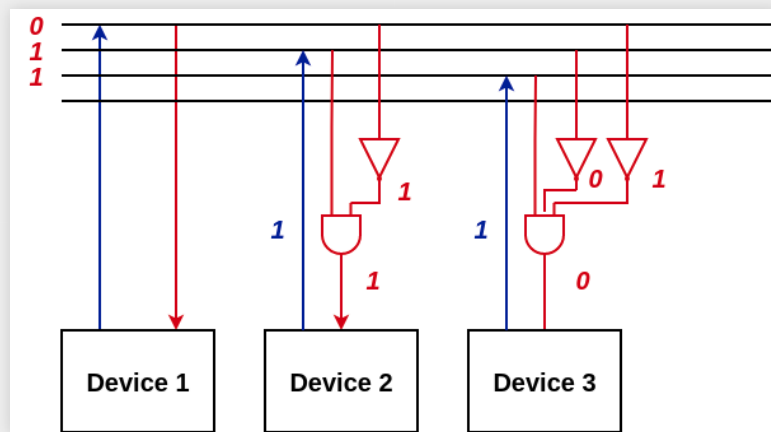
- 由一个中央仲裁根据优先级进行仲裁
- 所有设备和中央仲裁之间有两个控制线，分别发送请求和接收控制许可
- 优先级既可能是确定的，也可能是可编程的



基于自检测的分布式仲裁

基于自检测的分布式仲裁

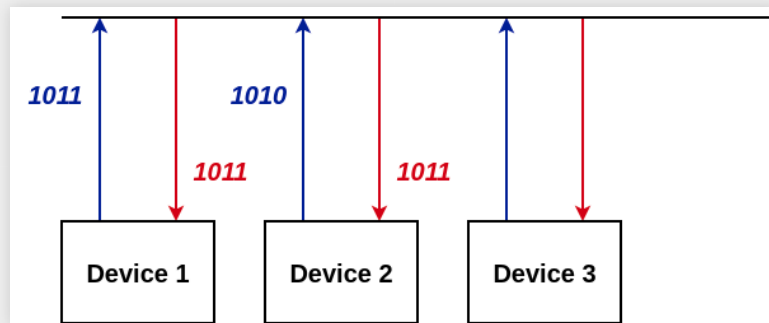
- 不需要中央仲裁器
- 每个设备有一个独立的控制线用于发送请求
- 按优先级从高到低排列，第 $i+1$ 个设备监听了前 i 个设备的请求控制线
- 若第 i 个设备发送了请求，当前 $i-1$ 个设备都没有发送请求时，该设备获得总线控制许可



基于冲突检测的分布式仲裁

基于冲突检测的分布式仲裁

- 不需要中央仲裁器
- 所有设备共享同一个控制线
- 如果检测到冲突，则重新发送请求，否则发送请求的设备获得控制许可
- 不同的标准可能采用不同的冲突检测方法



时钟与CPU时间

$$\text{CPU Time} = \frac{\text{seconds}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \times \frac{\text{avg. cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}}$$

CPU时间计算公式

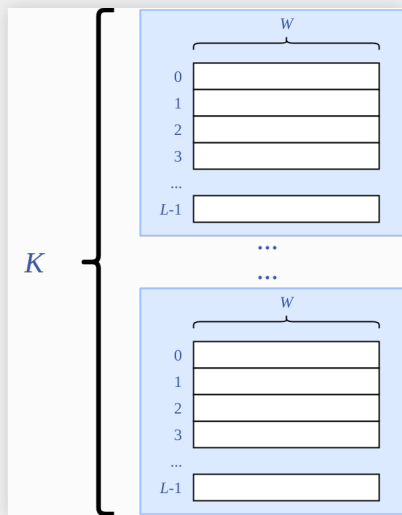
- 如何指导ISA的设计
- 如何解释CPU时间计算公式

来源: Linda Null, Julia Lobur, *The Essentials of Computer Organization and Architecture*, 4th Edition

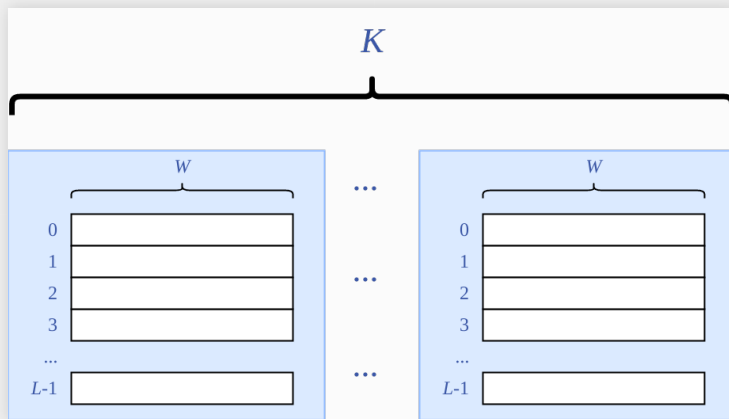
内存组织

- 按字寻址(word-addressable) & 按字节寻址(byte-addressable)
- 高位交叉编址 (High-order interleaving) & 低位交叉编址 (low-order interleaving)

高位交叉编址

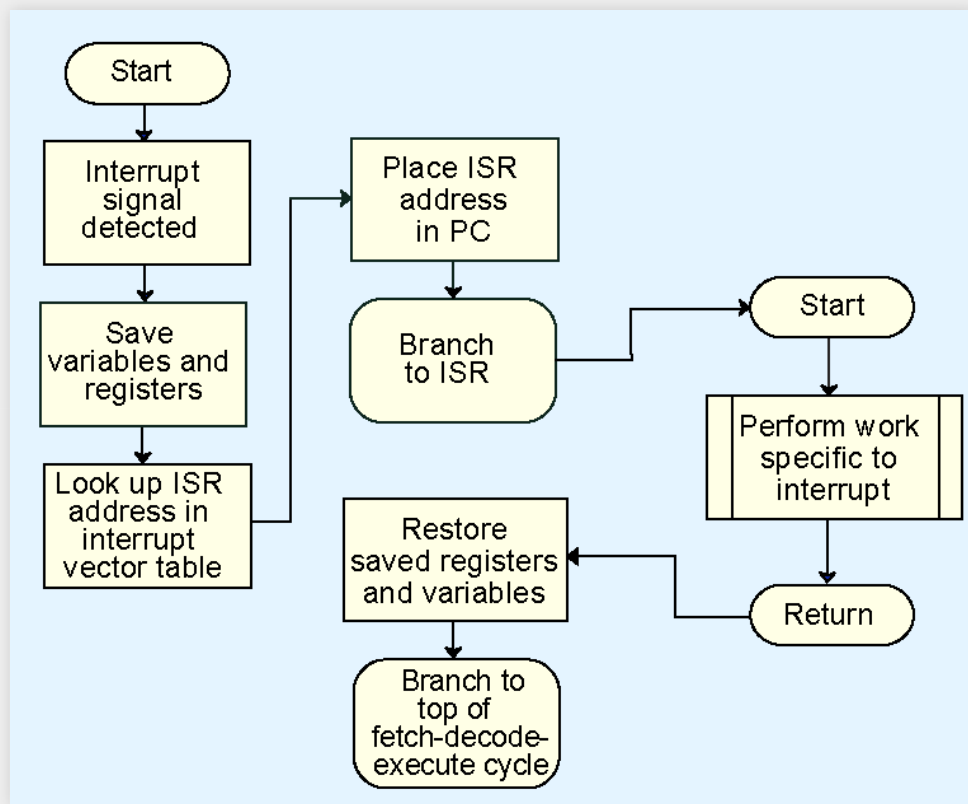


低位交叉编址



中断

- 中断的作用
- 处理中断的过程



本章总结

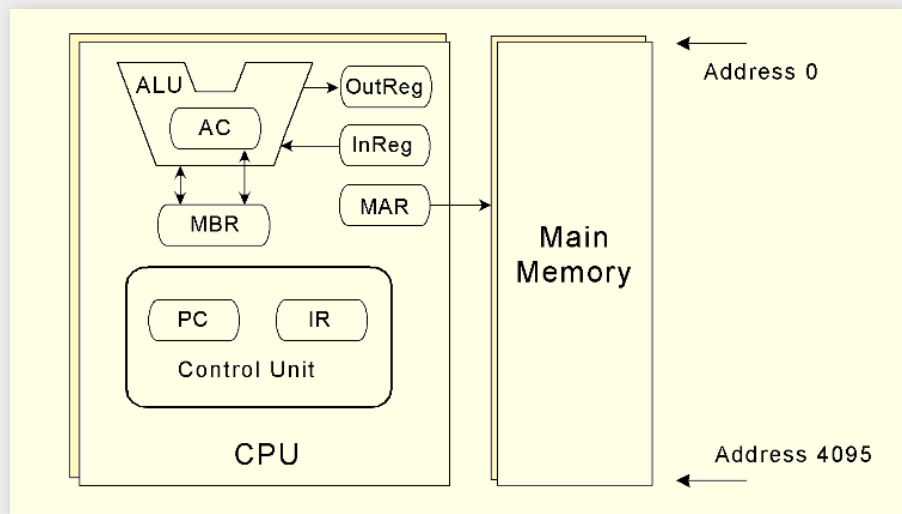
- CPU基本知识和组织结构
 - CPU的组件有哪些？
 - 硬连线控制和微程序控制的优点和缺点是什么？
- 总线仲裁
 - 不同总线仲裁的方式是如何工作的？
 - 各自的优缺点是什么？
- 内存寻址 & 编址
 - 什么是按字节寻址和按字寻址？它们有什么不同？
 - 不同的编址方式是如何工作的？
- 中断处理
 - 中断处理过程是什么？
 - 什么是中断处理程序和中断向量表？



MARIE

MARIE 体系结构

- 寄存器和 数据通路
- 指令集(指令的类型和编码、寻址方式)



来源: Linda Null, Julia Lobur, *The Essentials of Computer Organization and Architecture*, 4th Edition

寄存器传输表示

寄存器传输表示(Register Transfer Notation, RTN)或者寄存器传输语言(Register Transfer Language, RTL)可以对微操作进行形式化描述

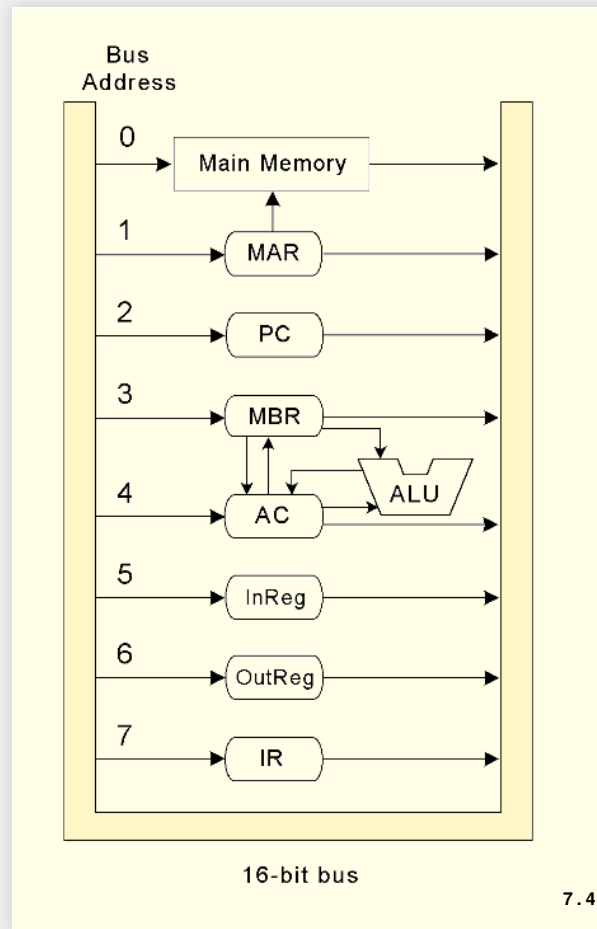
- $A \leftarrow B$: 将 B 中的值写入 A
- 变量 A 和 B 包括下面几类
 - 寄存器: 用寄存器名字表示, 例如: AC 、 MBR
 - 内存存储单元: 用 $M[y]$ 表示地址为 y 的值所对应的存储单元, 例如: $M[MAR]$
 - 指令中的操作数: 用 x 表示
 - 某个变量中的特定位: 用变量名加上比特位范围表示, 例如: $IR[11-0]$
 - 常数: 用数字表示, 例如: $0x01$, 1
 - 运算结果: 用算术运算的表达式表示, 例如: $AC + MBR$

MARIE中的控制信号

为了保证CPU的正确运行，需要通过控制信号 (Control signals) 来协调各部件的功能

MARIE架构中需要哪些控制信号？

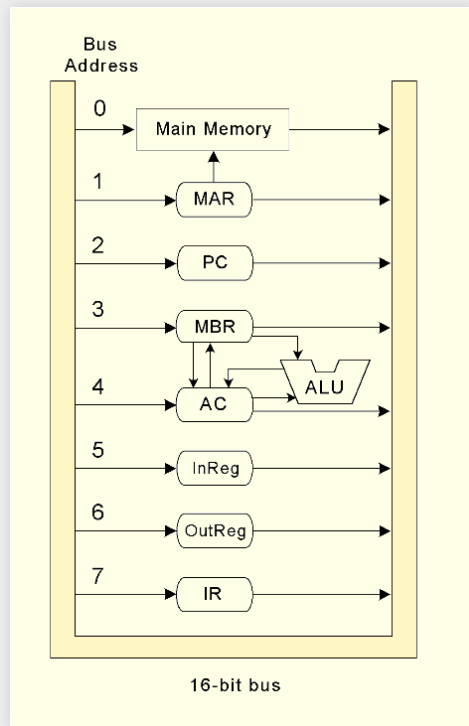
- 总线：控制总线的读和写
- ALU：控制ALU中的多路复用器
- 时钟：每个指令的完整执行需要多个时钟周期，需要根据当前的指令和对应的时钟周期计数决定对应的微操作信号
- 内存：控制内存的读写模式
- PC等具有替代数据源的设备：决定输入源
- 输入输出：输入、输出是否已经完成



确定控制信号

- 根据寄存器传输表示确定控制信号

示例1: LOAD x指令



微操作

信号模式 (只包含为1的信号)

$MAR \leftarrow PC$

$T_0 | P_1 | P_3$

$IR \leftarrow M[MAR]$

$T_1 | M_R | P_5 P_4 P_3 | \text{IncrPC}$

$PC \leftarrow PC + 1$

$x \triangleq IR[11-0],$

$T_2 | P_2 P_1 P_0 | P_3$

$OP \leftarrow \text{DECODE}(IR[15-12]),$

$MAR \leftarrow x$

$MBR \leftarrow M[MAR]$

$T_3 | M_R | P_4 P_3$

$AC \leftarrow MBR$

$T_4 | C_r | L_{alt}$

程序踪迹

程序踪迹(Program Trace):描述程序的运行状态

- 第一行写出程序运行前的寄存器和各内存单元取值
- 对RTN中的每一步, 写出运行后的寄存器和各内存单元取值

Step	RTN	PC	IR	MAR	MBR	AC
(initial values)		100	-----	-----	-----	-----
Fetch	MAR ← PC	100	-----	100	-----	-----
	IR ← M[MAR]	100	1104	100	-----	-----
	PC ← PC + 1	101	1104	100	-----	-----
Decode	MAR ← IR[11-0]	101	1104	104	-----	-----
	(Decode IR[15-12])	101	1104	104	-----	-----
Get operand	MBR ← M[MAR]	101	1104	104	0023	-----
Execute	AC ← MBR	101	1104	104	0023	0023

LOAD 104

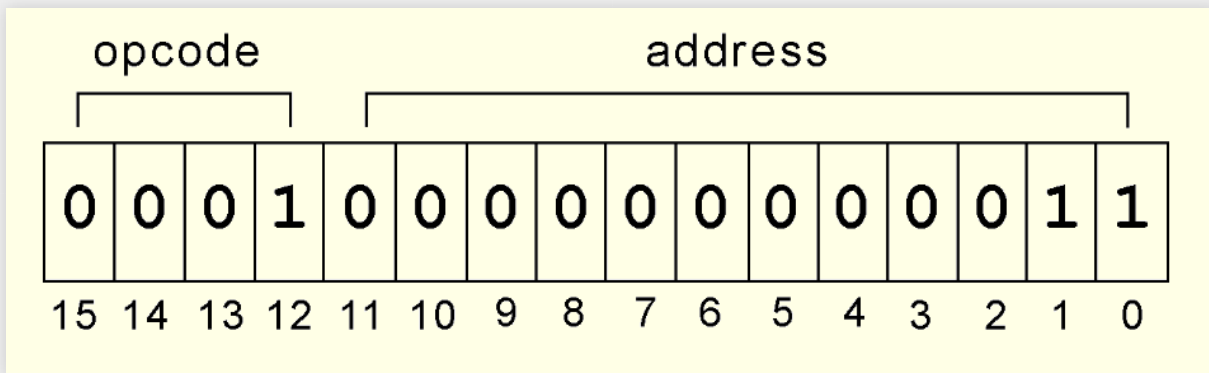
Step	RTN	PC	IR	MAR	MBR	AC
(initial values)		101	1104	104	0023	0023
Fetch	MAR ← PC	101	1104	101	0023	0023
	IR ← M[MAR]	101	3105	101	0023	0023
	PC ← PC + 1	102	3105	101	0023	0023
Decode	MAR ← IR[11-0]	102	3105	105	0023	0023
	(Decode IR[15-12])	102	3105	105	0023	0023
Get operand	MBR ← M[MAR]	102	3105	105	FFE9	0023
Execute	AC ← AC + MBR	102	3105	105	FFE9	000C

ADD 105

图片来源: Linda Null, Julia Lobur, *The Essentials of Computer Organization and Architecture*, 4th Edition

汇编语言

- 采用二进制表示的指令称为**机器指令**：(0001 0000 0000 0011)
- 采用符号表示的指令称为**汇编语言指令**(Assembly): **LOAD 3**
- 对应表示操作码的符号称为**助记符**(Mnemonic Instruction): **LOAD**



汇编指令**LOAD 3**对应的二进制指令编码

本章总结

体系结构

- 体系结构(寄存器、数据通路、内存分布)
- 指令集(编码、内存寻址)
- 控制：寄存器传输表示/寄存器传输语言
 - RTN的描述形式是怎样的？
 - 如何生成信号？

编程工具

- 程序踪迹
- 汇编语言
 - 两次通读是如何工作的？

深入理解计算机指令集

多字节数据的顺序

- 大端(big endian):
 - 更加自然：字节顺序和地址顺序一致
 - 可以快速确定数字的符号(符号位在最低地址)
 - 字符串的存储顺序和整数一致： "ABC" = 0x41 0x42 0x43
=> 0x414243
- 小端(little Endian):
 - 高位数字转为低位数字更方便：32位数字0x12345678和低16位0x5678的地址是一样的

指令集所代表的计算模型

按照CPU中数据存储的方式，指令集所代表的计算模型可以分为：

- 堆栈型架构 (stack architecture)
- 累加器型架构 (accumulator architecture)
- 通用寄存器型架构 (General-purpose register architecture, GPR)

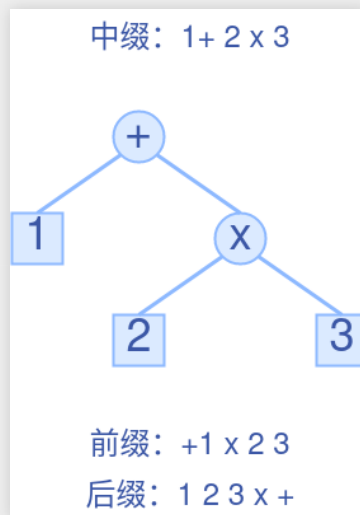
指令操作数的数量与计算模型的对应关系

不同存储方式支持的操作数数量，对应不同的计算模型：

存储方式	堆栈型	累加器型	通用寄存器型
零地址指令	是	是	是
单地址指令	是	是	是
双地址指令	否	否	是
三地址指令	否	否	是

逆波兰式

- 表达式的三种类型描述: 前缀表达式 (prefix)、中缀表达式 (infix)、后缀表达式 (postfix)
- 在不同表达式之间转换



指令编码

定长编码和变长编码

- 定长指令的译码相对容易但是浪费了存储空间
- 定长指令通常可以更快执行
- 变长指令译码复杂
- 不同类型的指令需要的操作数数量不同

定长操作码和扩展操作码

扩展操作码

- 使用位模式判断是否采用 L 位可以编码一个使用扩展操作码的指令集
- 如何生成操作码

```
0000 R1    R2    R3    }  
...                               } 15 three-address codes  
1110 R1    R2    R3    }  
1111 - escape opcode  
1111 0000 R1    R2    }  
...                               } 14 two-address codes  
1111 1101 R1    R2    }  
1111 1110 - escape opcode  
1111 1110 0000 R1    }  
...                               } 31 one-address codes  
1111 1111 1110 R1    }  
1111 1111 1111 - escape opcode  
1111 1111 1111 0000 }  
...                               } 16 zero-address codes  
1111 1111 1111 1111 }
```

寻址方法

寻址方式代表指令集在内存中定位存储单元的方式，常见的寻址方法：

- 立即寻址(Immediate Addressing)
- 直接寻址(Direct Addressing)
- 间接寻址(Indirect Addressing)
- 变址寻址(Indexed Addressing)
- 基址寻址(Based Addressing)
- 堆栈寻址(Stack Addressing)

理论加速比分析

假设:

- 流水线有 s 个阶段
- 所有阶段都在一个时钟周期 t 内完成
- 各阶段执行时间都相等, 并且等于时钟周期 t

理论加速比

对于单周期、等长多阶段流水线:

$$\text{理论加速比} = \lim_{n \rightarrow +\infty} \frac{ns}{s + n - 1} = s$$

在该情况下:

- 理论加速比等于流水线级数

(理论) 加速比分析

假设（更实际）：

- 流水线有 s 个阶段
- 所有阶段都在一个时钟周期 t 内完成
- 各阶段执行时间~~不都相等~~，第 i 个阶段需要 t_i
- 程序具有 n 个指令

隐含条件:时钟周期大于或等于各阶段执行时间

$$t \geq t_i, \forall 1 \leq i \leq s \Rightarrow t \geq \max_{1 \leq i \leq s} \{t_i\}$$

(理论) 加速比分析

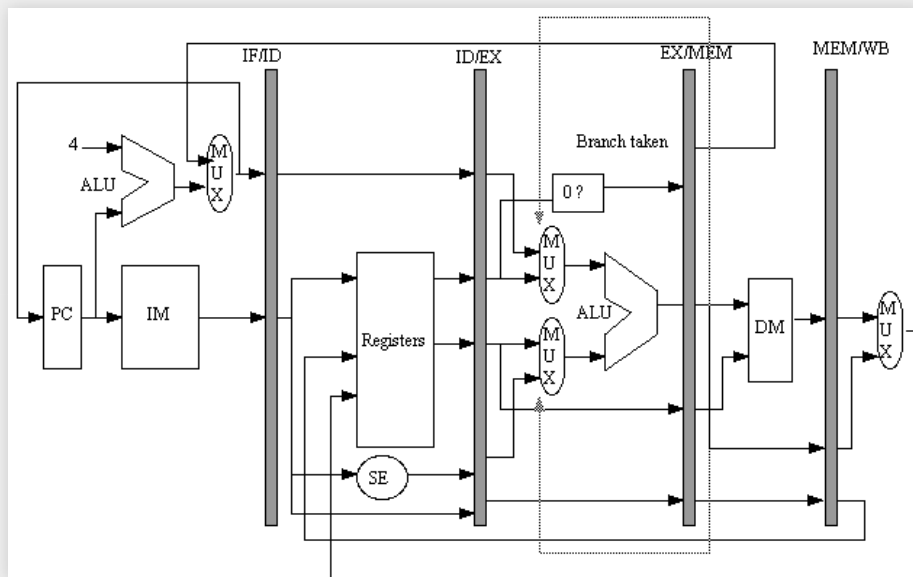
加速比为

$$\text{加速比} = \frac{n(\sum_{i=1}^s t_i)}{st + (n-1)t} \leq \frac{n(\sum_{i=1}^s t_i)}{(s+n-1) \max_{1 \leq i \leq s} \{t_i\}}$$

理论加速比为

$$\begin{aligned} \text{理论加速比} &= \lim_{n \rightarrow +\infty} \frac{n(\sum_{i=1}^s t_i)}{st + (n-1)t} = \frac{\sum_{i=1}^s t_i}{t} = \frac{\sum_{i=1}^s t_i}{\max_{i=1}^s t_i} \\ &\leq \frac{\sum_{i=1}^s t_i}{\max_{1 \leq i \leq s} \{t_i\}} \end{aligned}$$

结构冒险



MIPS

- 32位、按字节寻址的指令集
- 可以采用5阶段流水线实现
 - IF, ID, EX, MEM, WB

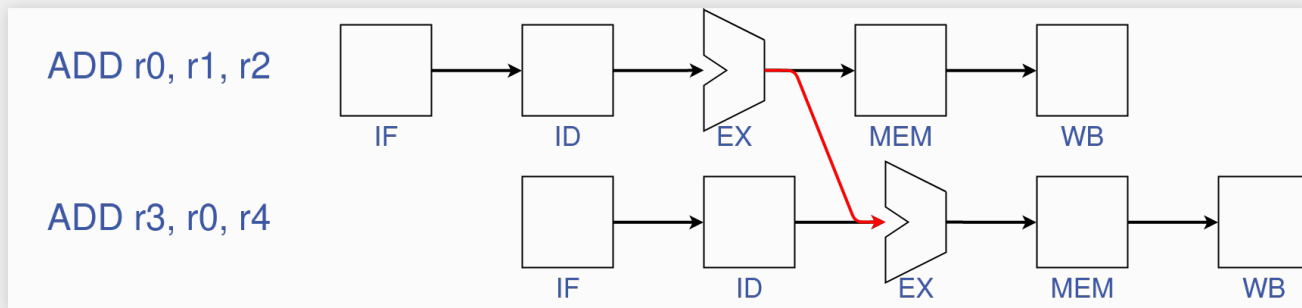
RAW数据冒险

数据冒险(Data Hazard): 指令之间存在数据依赖

- 例: MIPS指令间存在读-写依赖(Read-after-Write, RAW)

```
I1: ADD r0, r1, r2  <---- r0 = r1 + r2  
I2: ADD r3, r0, r4  <---- r3 = r0 + r4
```

- 解决方案: 数据定向/数据旁路 (Data forwarding/bypassing):
从EX/MEM的交叉电路添加一条特殊回路作为ALU的输入



加载-使用型数据冒险

- 第二类数据冒险称为加载-使用型数据冒险(Load-use data hazard)
- MIPS指令集中加载 - 使用型数据冒险的例子:

```
I1: LOAD r0, x      <----- r0 = [x]  
I2: ADD  r2, r0, r1  <----- r2 = r0 + r1
```

- 解决方法: 插入一条NOP指令, 阻塞(Stalling)流水线

控制冒险

控制冒险(Control Hazard): 指令直接存在控制依赖, 前一条指令的结果决定了后一条指令是否需要执行

- 主要由于分支指令造成, 因此也被称为**分支冒险(Branch Hazard)**

```
I1: BEQ r1, r2, r0 <--- 如果r1=r2, 跳转到r0对应的地址 (Ik)
I2: ADD r1, r2, r3 <--- r1 = r2 + r3
...
Ik: ADD r1, r2, r4 <--- r1 = r2 + r4, r0对应的指令
```

解决办法1: 阻塞流水线

解决办法2: **分支预测(Branch prediction)**

本章总结

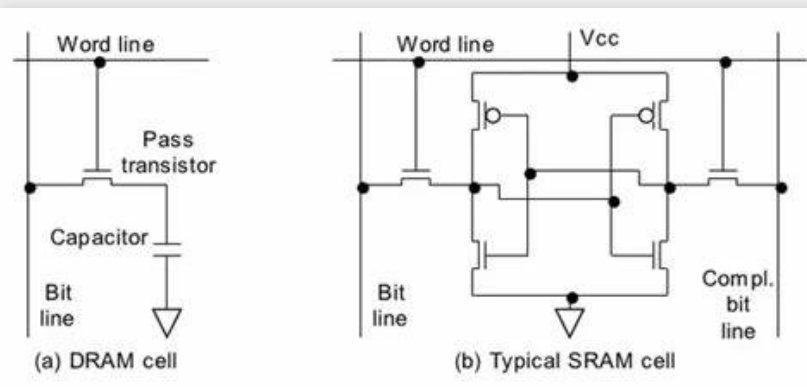
- 字节序
 - 什么是大端和小端？它们各自有什么优缺点？
- 计算模型
 - 堆栈型架构是如何工作的？如何将表达式转换为逆波兰表达式？
- 指令编码
 - 如何使用扩展操作码？
- 寻址方法
 - 数据最终存储位置
 - 用途
- 流水线
 - 如何分析流水线加速比？
 - 什么是冒险以及如何解决？

存储器

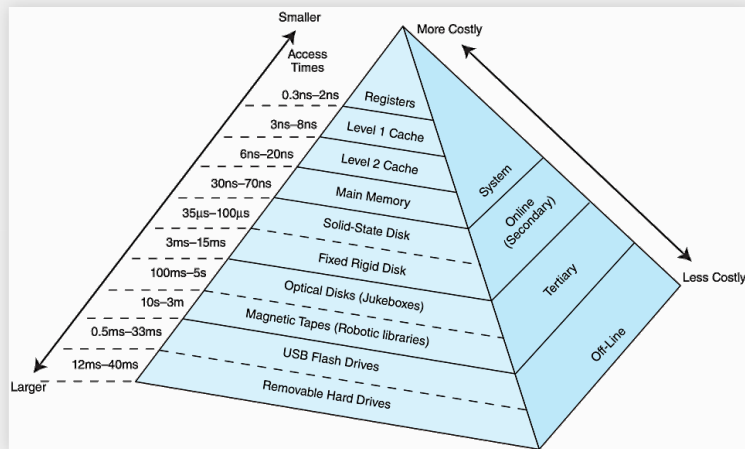
存储器基本知识

按照功能划分

- 只读存储器(Read-Only Memory, ROM)
- 随机访问存储器 (Random Access Memory, RAM)
 - 静态随机访问存储器 (Static RAM, SRAM)
 - 动态随机访问存储器 (Dynamic RAM, DRAM)



数据存储的层次化组织



基本思路: 由计算机存储层次结构 (storage hierarchy) 中的多级存储介质构成层次化的存储系统 (hierarchical storage system) 以达到

- 更快的平均访问速度
- 低成本的大存储容量

有效访问时间的通用分析方法

对一个层次化存储系统，我们假设：

- 该层次化存储系统由N级存储构成（编号从1到N）
- 第i级命中率为 h_i
- 第i级的访问时间为 t_i

有效访问时间有两类情况：

- S_i 事件：数据在第i级存储查找成功
 - 隐含条件：数据在前i-1级存储查找失败
 - 事件概率： $h_i \left[\prod_{j=1}^{i-1} (1 - h_j) \right]$
- F 事件：数据没有在任何一级存储查找成功
 - 事件概率： $\prod_{j=1}^N (1 - h_j)$

并行访问的有效访问时间分析

对于采用并行访问的N级层次化存储器

- S_i 事件的访问时间: $t_i = \max_{j=1}^i t_j$
- S_i 事件的概率: $h_i \left[\prod_{j=1}^{i-1} (1 - h_j) \right]$
- F 事件的访问时间: $t_N = \max_{j=1}^N t_j$
- F 事件的概率: $\prod_{j=1}^N (1 - h_j)$

根据期望计算公式，并行访问的有效访问时间为：

$$\text{EAT} = \sum_{i=1}^N \left[\left[\prod_{j=1}^{i-1} (1 - h_j) h_i \right] t_i \right] + \left[\prod_{j=1}^N (1 - h_j) \right] t_N$$

串行访问的有效访问时间分析

对于采用串行访问的N级层次化存储器

- S_i 事件的访问时间: $\sum_{j=1}^i t_j$
- S_i 事件的概率: $h_i \left[\prod_{j=1}^{i-1} (1 - h_j) \right]$
- F 事件的访问时间: $\sum_{j=1}^N t_j$
- F 事件的概率: $\prod_{j=1}^N (1 - h_j)$

$$\text{EAT} = \sum_{i=1}^N \left(\prod_{j=1}^{i-1} (1 - h_j) h_i \right) \cdot \left(\sum_{j=1}^i t_j \right) + \left(\prod_{j=1}^N (1 - h_j) \right) \cdot \left(\sum_{j=1}^N t_j \right)$$

重新组织后得到

$$\text{EAT} = \sum_{i=1}^N \left(\prod_{j=1}^{i-1} (1 - h_j) \right) t_i$$

存储器的数据局部性

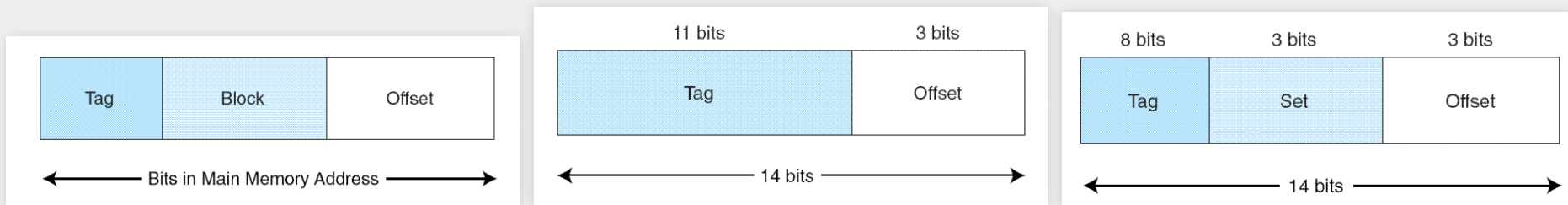
- 为了减少访问失效的概率，存储系统需要对未来可能访问的数据进行预测，局部性是一类有效的预测方法。

三类常见的局部性：

- 时间局部性(temporal locality)：最近访问的数据在不远的将来很有可能会再次访问
- 空间局部性(spatial locality)：即将访问的数据和最近访问的数据地址空间接近
- 顺序局部性(sequential locality)：数据按顺序访问

缓存映射策略

- 决定了缓存中数据块的组织方式
- 决定了如何从内存地址获得缓存块的索引
- 三类映射策略：直接映射(Direct mapping)、全相联映射(Fully associative mapping)和组相联映射(Set associative mapping)



来源: Linda Null and Julia Lobur, *Computer Organization and Architecture*, 4th Edition

缓存替换策略

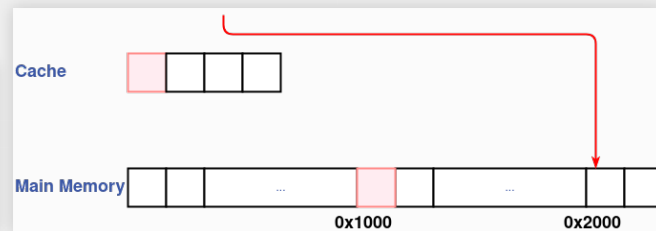
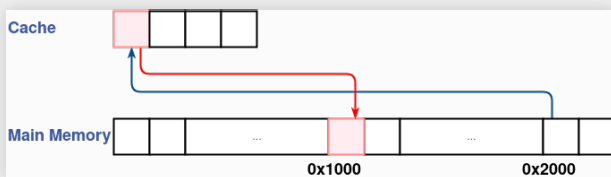
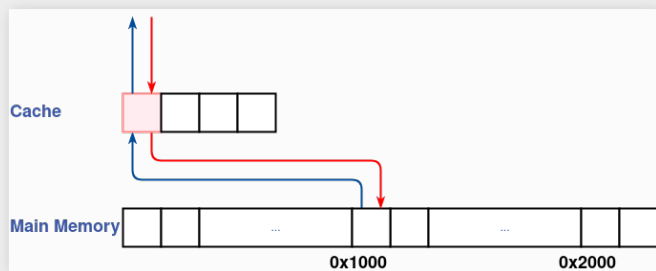
- 缓存替换策略的目标：提高未来数据访问的命中率
- 理论上最优的方法：总是**换出** (evict)未来访问频率最低的数据块

常见的缓存替换策略：

- **最久未使用** (Least Recently Used, LRU)
- **先入先出** (First-in First-out, FIFO)
- **随机替换** (Random)

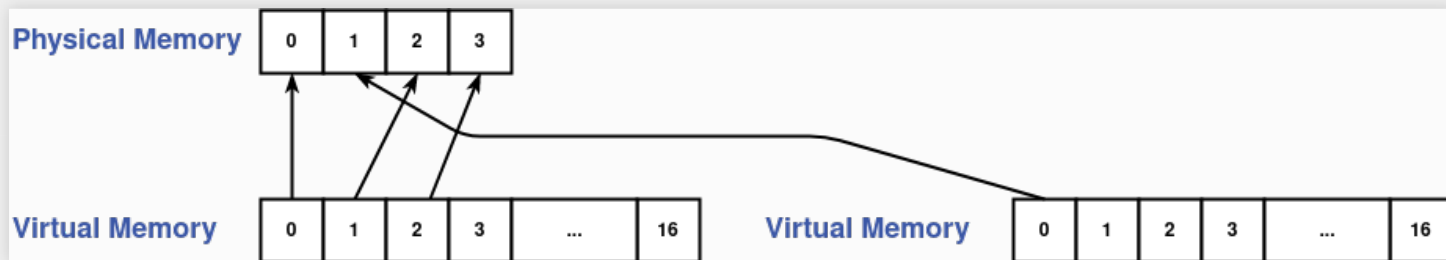
缓存写策略

- 两类写事件
 - 脏块
 - 写缺失
- 缓存写策略
 - 脏块上的写策略：直写 (Write-through) & 回写 (Write-back)
 - 写缺失上的写策略：写分配策略 (Write-Allocate) & 写不分配策略 (No-Write-Allocate)



虚拟内存 (1)

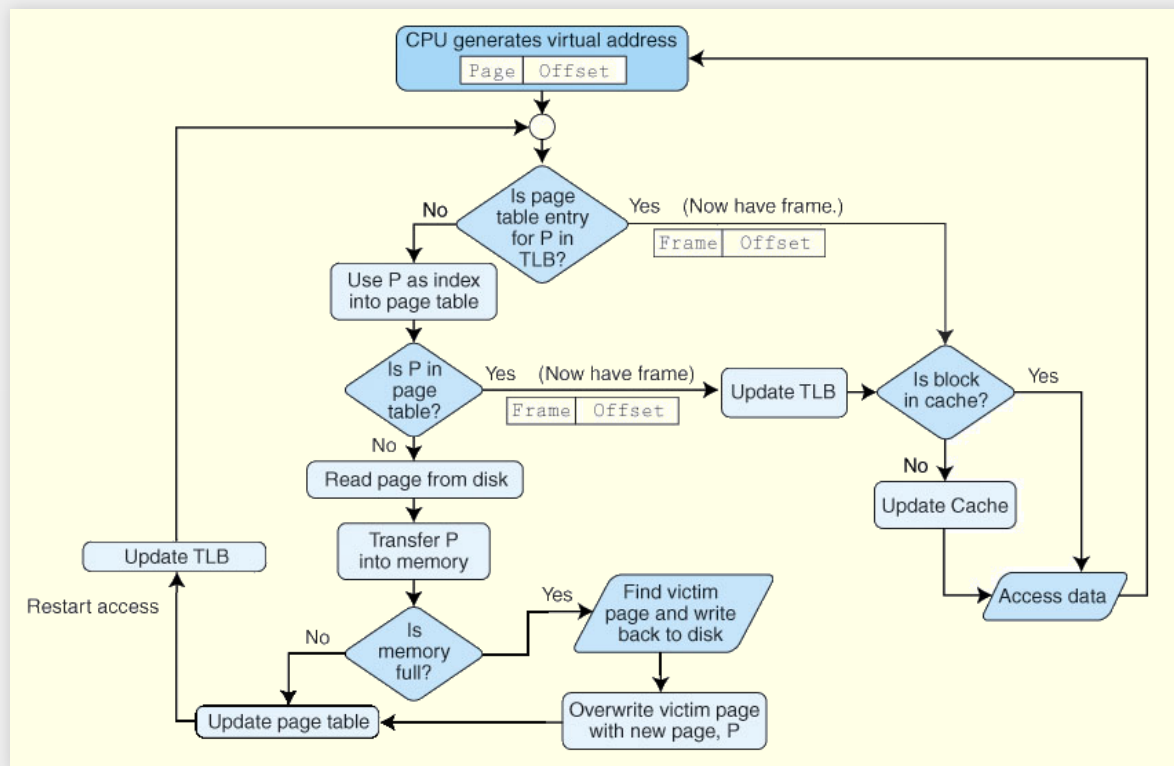
- 分页 & 页表 (page table)
- 地址转换
 - 虚拟地址到物理地址
 - 页编号(page number)到页面编号(frame number)



进程 1	虚页编号	页面编号	有效位	进程 2	虚页编号	页面编号	有效位
	0	0	1		0	1	1
	1	2	1		1	-	0
	2	3	1		2	-	0
	...	...	...		...	...	...
	16	-	0		16	-	0

虚拟内存(2)

- 转换后备缓冲器 (Translation Lookaside Buffer, TLB)
- 内存访问的过程和访问时间



内存管理

- 分页 & 分段
- 内部碎片 & 外部碎片

进程1	8K	0	进程1	S1	8K	0
进程2	10K	1		S2	10K	1
进程3	9K	2		S3	9K	2
进程4	4K	3	进程2	S1	4K	3
		4		S2	11K	4
		5				5
		6				6
		7				7

本章总结

- 层次化存储
 - 如何分析层次化存储系统的有效访问时间？
 - 什么是局部性？不同类型的局部性的含义是什么？
- 缓存
 - 不同类型的缓存映射策略是如何工作的？
 - 什么是缓存替换策略？
 - 什么是缓存写策略？
- 虚拟内存
 - 什么是页表？内存地址转换是如何实现的？
 - 什么是TLB？
 - 掌握内存寻址的完整工作流程
- 内存管理
 - 分段是如何工作的？
 - 什么是内部和外部碎片？什么导致了这些碎片的产生？



I/O

阿姆达尔定律

$$S = \frac{1}{(1 - f) + \frac{f}{k}}$$

- f : 优化的部分占总体性能的比例
- k : 优化的部分的加速比
- S : 系统的整体加速比

阿姆达尔定律定量描述了提高局部性能对整体性能的影响

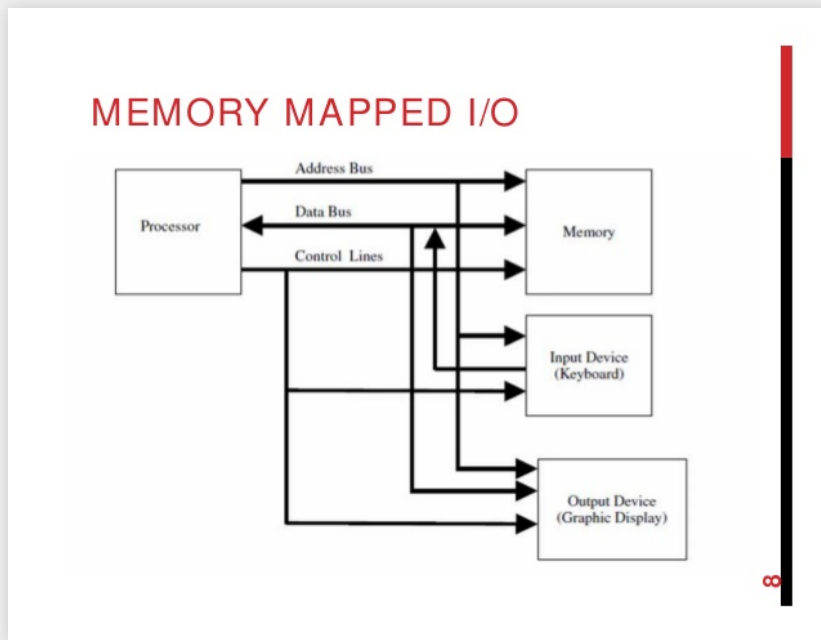
I/O 控制方法

有五种常见的 I/O 控制方法

- 程序控制 I/O(Programmed I/O): 用特殊寄存器保存 I/O 设备的状态, CPU 轮询寄存器状态决定是否需要处理 I/O 事件
- 中断驱动 I/O(Interrupt-driven I/O): I/O 设备触发系统中断, 由中断处理程序处理 I/O 事件
- 内存映射 I/O(Memory-mapped I/O): 设备中的数据映射为内存地址空间的一部分
- 直接内存访问 I/O(Direct Memory Access, DMA): 采用专用芯片处理 I/O 设备和主存之间的数据传输
- 通道 I/O(Channel I/O): 采用专用芯片, 将多个 I/O 路径合并为 I/O 通道

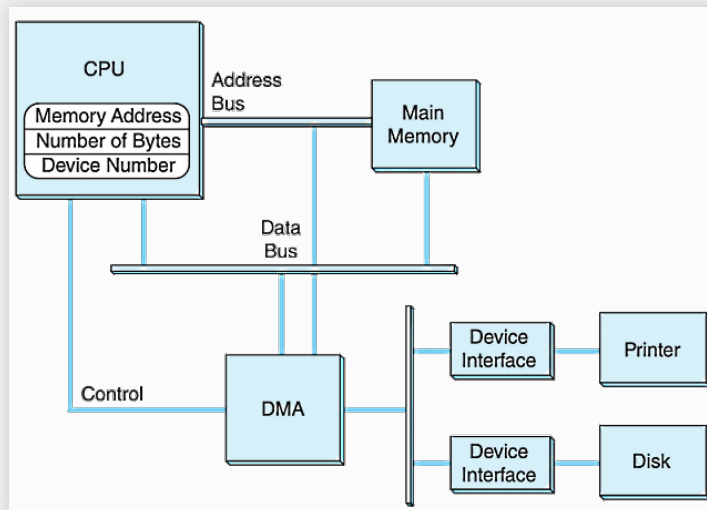
理解它们的工作方式和特性

内存映射 I/O



- I/O 设备与内存共享地址空间 -I/O 设备内部的存储映射到内存的地址空间
- CPU 通过访问内存的方式直接访问 I/O 设备的内部存储

直接内存访问

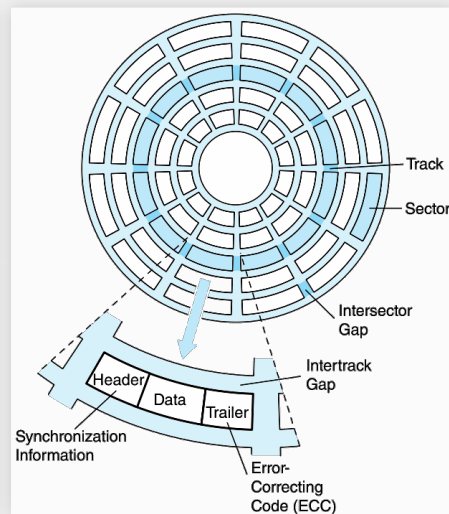
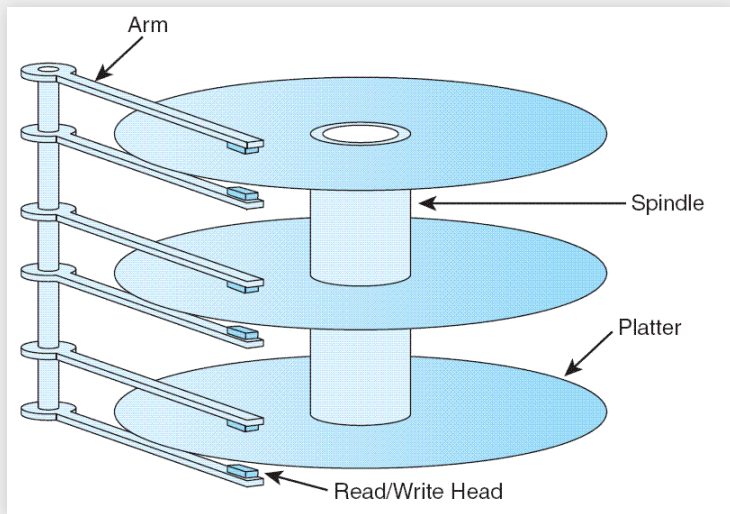


- 设备到内存的数据传输由**DMA**单元处理
- CPU不复制数据，而是指定**传输信息**

来源: Linda Null and Julia Lobur, *Computer Organization and Architecture*, 4th Edition

机械硬盘

- 数据块由圆柱面(cylinder)、盘面(surface)和扇区(sector)进行寻址
- 访问时间 = 寻道时间 + 旋转延迟
- 传输时间 = 访问时间 + $\frac{\text{数据大小}}{\text{传输速率}}$



其它持久性存储

- 固态硬盘
- 光盘
 - CD-ROM/CD-RW/DVD/蓝光光盘
- 磁带
- 理解它们的工作方式和特性(存储、可读写容量、价格等)

RAID

按照磁盘阵列中数据的组织方式分类：

- RAID level 0或者RAID-0
- RAID-1
- RAID-2
- RAID-3
- RAID-4
- RAID-5
- RAID-6
- RAID-DP

按照层次化方式进行组合：

- RAID-01
- RAID-10

本章总结

- 阿姆达尔定律
 - 如何使用公式计算未知变量？
- I/O控制
 - 不同的I/O控制方式是如何工作的？
- 持久化存储介质
 - 硬盘是如何工作的，如何计算EAT？
 - 不同的持久数据存储有哪些优点和缺点？
- RAID
 - RAID-0到RAID-6的工作方式是怎么样的？
 - 不同级别的RAID的特性是什么？

A horizontal banner featuring a traditional Chinese architectural scene. The left side is a solid red overlay, while the right side shows a photograph of a temple roof with ornate, curved eaves and decorative finials. The text '系统软件' is centered in white.

系统软件

保护环境

- 保护环境是由（主机）操作系统或虚拟机监控程序保护的环境
- 资源、进程和数据被隔离
- 实现受保护环境的常见技术
 - 虚拟机 (Virtual Machine, VM)
 - 子系统 (Subsystem)
 - 逻辑分区 (Logical Partitioning, LPAR)

编程工具

- 三个阶段: 编译时 (Compile time) , 加载时(Load time), 运行时(Run time/or Runtime)
- 理解它们的作用
 - 汇编器(Assembler)
 - 编译器(Compiler)
 - 加载器(Loader)
 - 链接器(Linker)

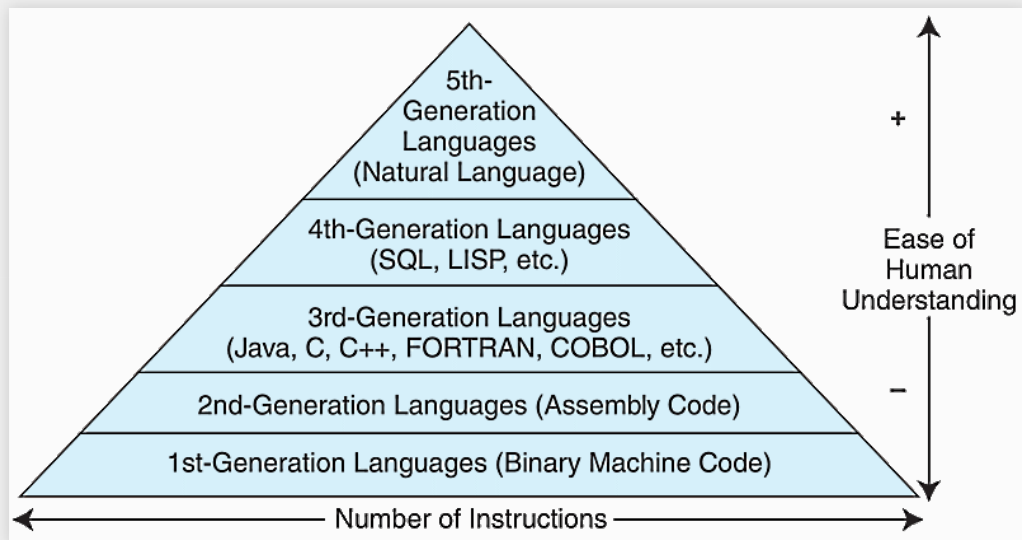
可重定位二进制代码的实现

- 每个指令/数据的地址和指令/数据的引用被存储为程序起始的偏移量(偏移值)
- 当程序被加载到内存时, 基地址 (base address) 就是程序的起始地址
- 实际地址为基地址+偏移量(base + offset)。
- 重定位的实现方式
 - 使用基址寻址模式 (base addressing mode)
 - 使用加载时地址重写 (rewriting at load time)

链接方式对比

	静态链接	加载时动态链接	运行时动态链接
阶段	编译时	加载时	运行时
二进制文件大小	大	小	小
加载速度	快	慢 (冷启动)	快
调用速度	快	快	慢 (冷启动)
依赖更新	重链接	重新执行	程序内更新函数指针

不同阶段的编程语言

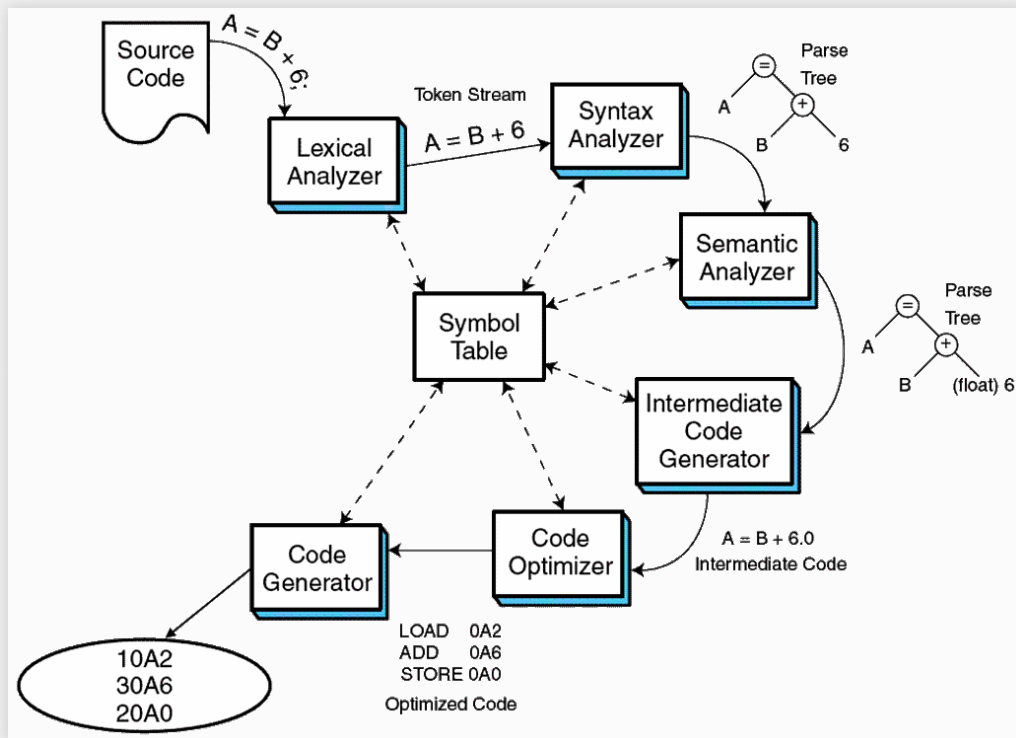


- 汇编是典型的第二代编程语言
- C/C++, Fortran, Java是典型的第三代编程语言

Source: Linda Null, Julia Lobur, *The Essentials of Computer Organization and Architecture*, 4th Edition

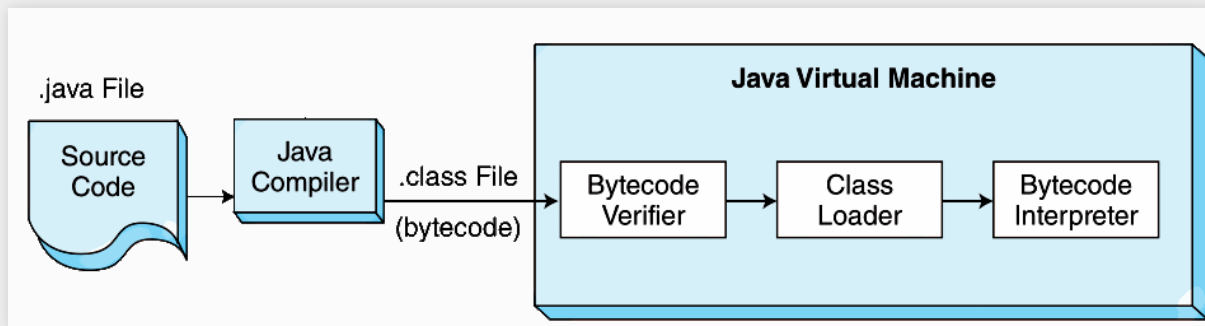
编译器

- 编译器将高级语言翻译成低级语言，或直接翻译成机器码
- 该过程被称为**编译(compilation)**，通常包含六个阶段



JAVA

- Java的最终编译目标是Java虚拟机 (Java Virtual Machine, JVM)
- JVM拥有自己的指令集, 即Java字节码 (Java Bytecode)
- Java 编译器将 Java 代码编译为 Java 字节码, 而不是直接编译为机器码。



Source: Linda Null, Julia Lobur, *The Essentials of Computer Organization and Architecture*, 4th Edition

数据模式

- 数据库模式定义了数据库中数据的访问。
- 物理视图/模式 (Physical schema) 针对数据库开发者, 计算机对数据库的视图: 物理文件的位置、索引等。
- 逻辑视图/模式 (Logical schema) 针对数据的设计者, 特权用户对数据库的视图: 表、数据类型、字段大小等。
- 用户视图 (User view) 针对应用程序开发人员和/或最终用户。非特权用户对数据库的视图: 类似于逻辑视图但是有更严格的数据访问控制。

事务

- 事务管理 (Transaction management) 是现代数据库的重要功能
- 用于保证多用户并行处理时的系统一致性
- 事务管理通常具有如下四种性质 (简记ACID) :
 - 原子性 (Atomicity): 同一个事务中的操作要么全部执行, 要么全部不执行
 - 一致性 (Consistency): 所有的数据更新必须满足相关约束
 - 隔离性 (Isolation): 任意两个事务之间的执行是不会互相影响的(除非是按序执行的)
 - 持久性 (Durability): 成功的事务会写入持久存储介质

本章总结

- 操作系统
 - 有哪些实现保护环境的方式？
- 编程工具
 - 编译时、加载时、运行时的含义是什么？
 - 什么是可重定位代码？如何生成最终代码？
 - 什么是链接和绑定？
 - 在不同阶段可以使用的工具有哪些？
 - 编译包含哪些过程？
- 数据库
 - 什么是模式？
 - ACID对应的性质和含义是什么？

可选的体系结构

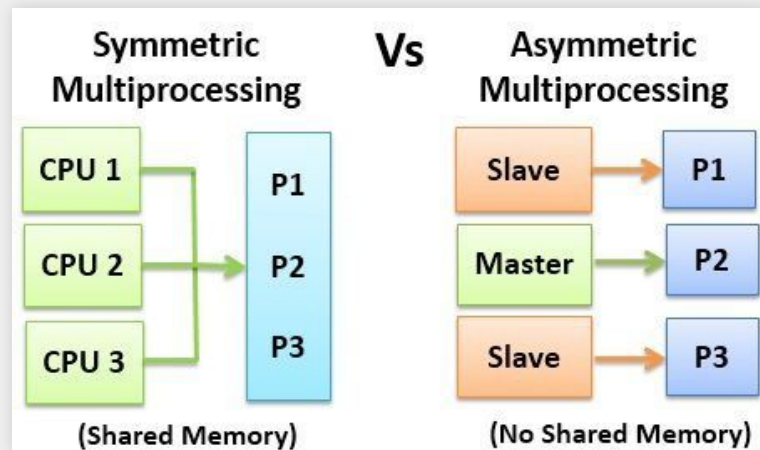
RISC & CISC

RISC	CISC
多寄存器集	单寄存器集
1条指令包含最多3个操作数	1条指令通常只有1个或2个操作数
通过寄存器窗口传递参数	通过内存栈传递参数
单周期指令，高度流水线化	多周期指令，流水线化程度低
硬连线电路实现	微代码控制
指令数量少，长度等长	指令数量多，长度不同
只有LOAD和STORE指令可以访问内存	许多指令都可以访问内存
寻址方法较少	支持多种寻址方法

费林分类法

- 四种基础的分类：
 - 单指令流、单数据流体系结构(Single instruction stream, single data stream,SISD)
 - 单指令流、多数据流体系结构 (Single instruction stream, multiple data stream,SIMD)
 - 多指令流、单数据流体系结构 (Multiple instruction stream, single data stream,MISD)
 - 多指令流、多数据流体系结构 (Multiple instruction stream, multiple data stream,MIMD)
- 按照是否采用共享内存进行处理器互联：
 - 对称多处理器 (Symmetric Multiprocessor, SMP)
 - 大规模并行处理器/非对称多处理器 (Massive Parallel Processor/Asymmetric Multiprocessor, MPP/AMP)

SMP & MPP



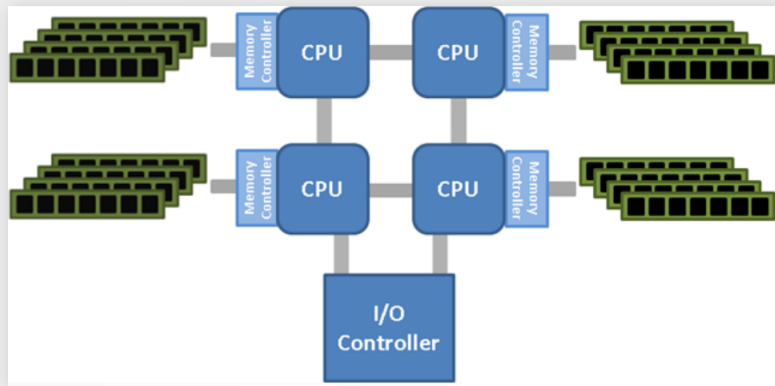
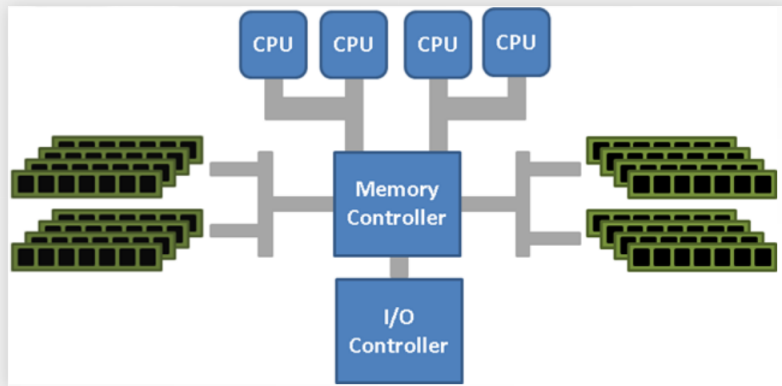
	SMP	MPP
处理器数量	较少	较多
访问内存方式	共享内存	分布式内存
处理器间通信	通过内存交换数据	通过互联网络交换数据

超标量、超长指令字、向量机

- 流水线、超标量、超长指令字: 指令集并行
- 向量机: SIMD
- 理解这些技术的工作方式和特性(性能提升、限制、案例)

UMA、NUMA、互连网络

- 理解统一内存访问 (Uniform Memory Access,UMA) 和非统一内存访问 (Non-uniform memory access, NUMA) 的特性 (内存分布、连接特性、内存访问时间、缓存一致性等)
- 理解交叉开关网络的特性和多阶段互连网络(开关的数量、阻塞/非阻塞等)



本章总结

- RISC & CISC
 - 寄存器窗口的结构是什么样的？如何分析寄存器窗口？
 - RISC和CISC的区别是什么？
- 费林分类及其扩展
 - 超标量、超长指令字和向量机是如何工作的？它们实现了何种类型的并行？
 - SMP和MMP之间的区别是什么？
 - UMA和NUMA之间的区别是什么？
 - 什么是交叉开关网络和多阶段互连网络？它们之间的区别是什么？

嵌入式系统

看门狗定时器

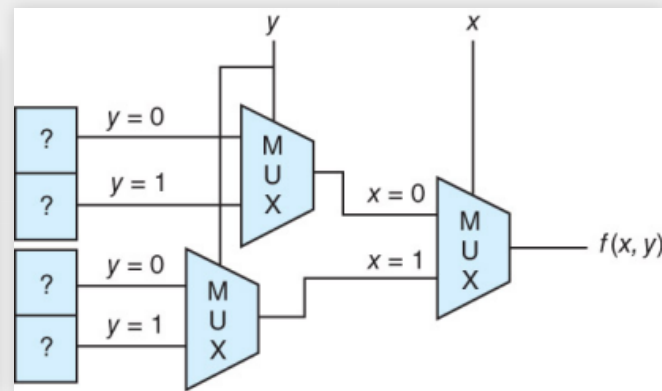
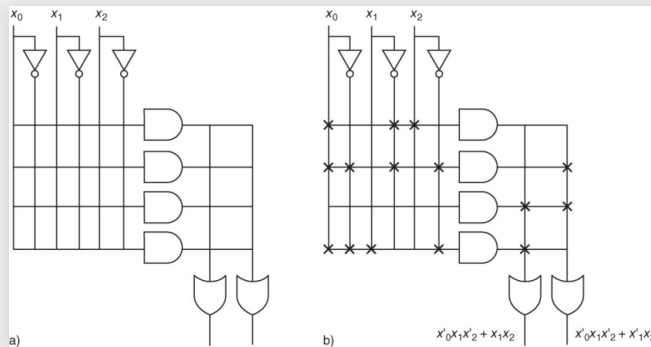
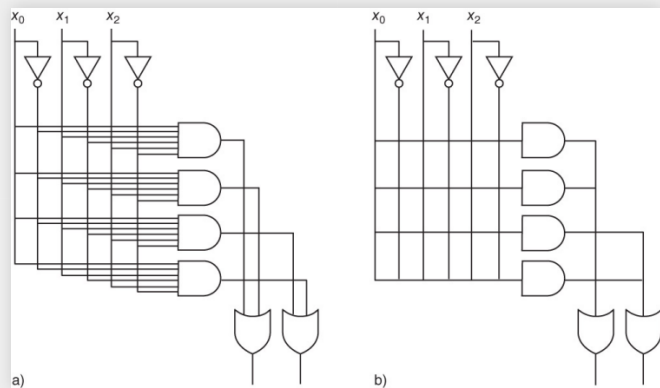
看门狗定时器提供了一种 **自动检测错误、系统重置** 的机制

- 当程序被调度执行时，计数器被设置为初始值
- 每隔一段（预定义）时间，看门狗定时器的计数器减1
- 程序执行完成后，重置看门狗定时器的计数器为初始值
- 计数器归零时重置整个系统

可编程逻辑设备

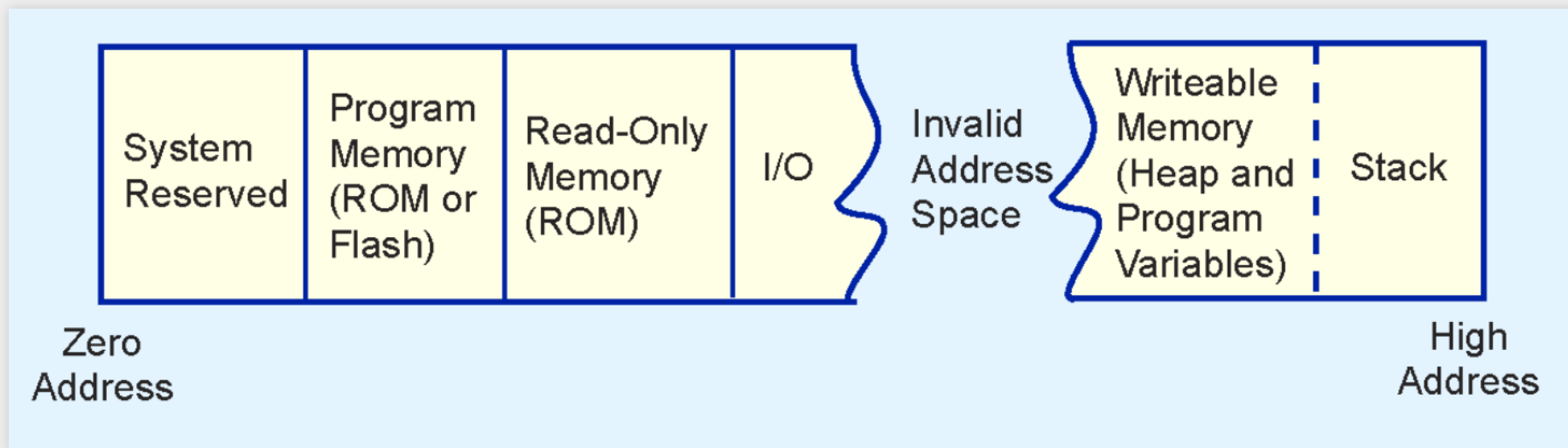
可编程逻辑设备 (Programmable Logic Device, PLD) 的三种实现方式:

- 可编程阵列逻辑 (Programmable Array Logic, PAL)
- 可编程逻辑阵列 (Programmable Logic Array, PLA)
- 可编程门阵列 (Field Programmable Gate Array, FPGA)



嵌入式系统的内存分布

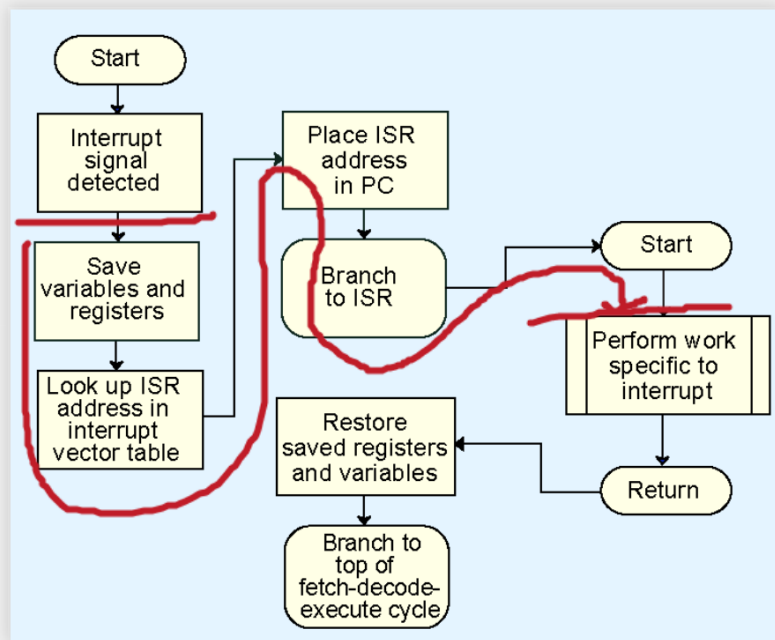
- 嵌入式系统中可能包含多种类型的内存
- 存储空间可能是不连续的
- 为了避免内存泄露，一些程序避免使用动态内存分配



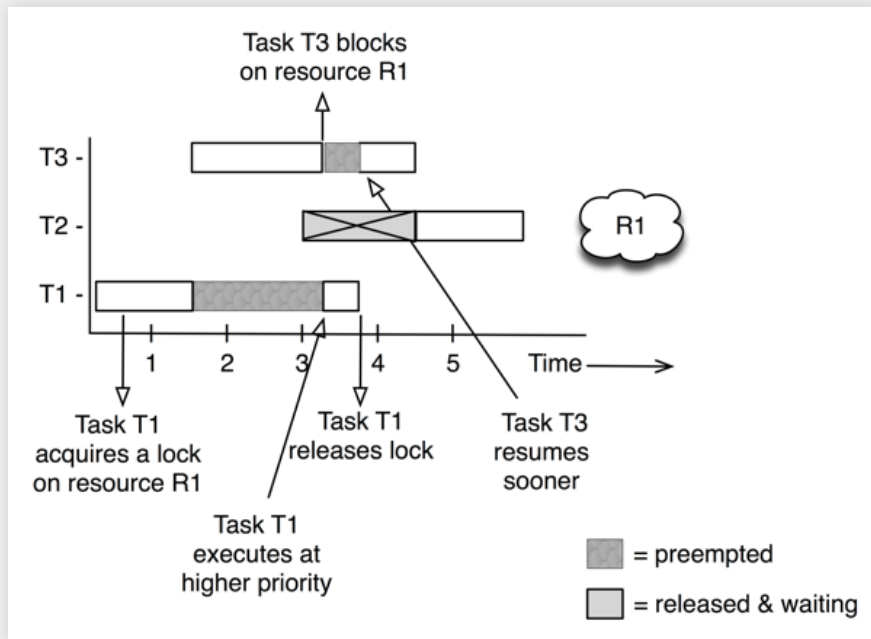
来源: Linda Null, Julia Lobur, *The Essentials of Computer Organization and Architecture*, 4th Edition

中断延迟

中断延迟是指中断发出时到中断处理程序（ISR）的第一条指令执行之间的时间。它是衡量响应性的关键指标。



优先级继承

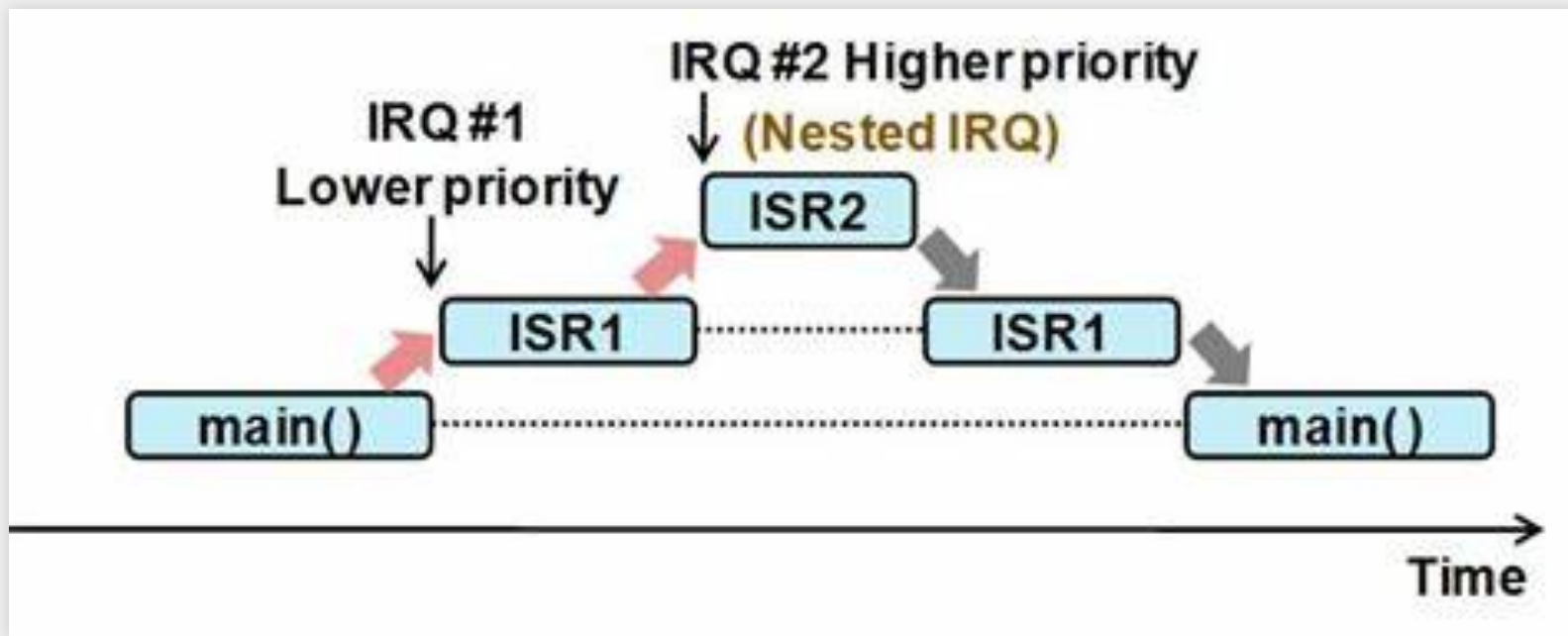


优先级继承(Priority Inheritance): 有依赖关系的任务将具有相同的优先级

来源: <https://www.drdobbs.com/jvm/what-is-priority-inversion-and-how-do-ya/230600008>

中断嵌套

- 低优先级任务屏蔽了高优先级任务的中断。
- **中断嵌套**: 允许高优先级抢占低优先级中断的执行



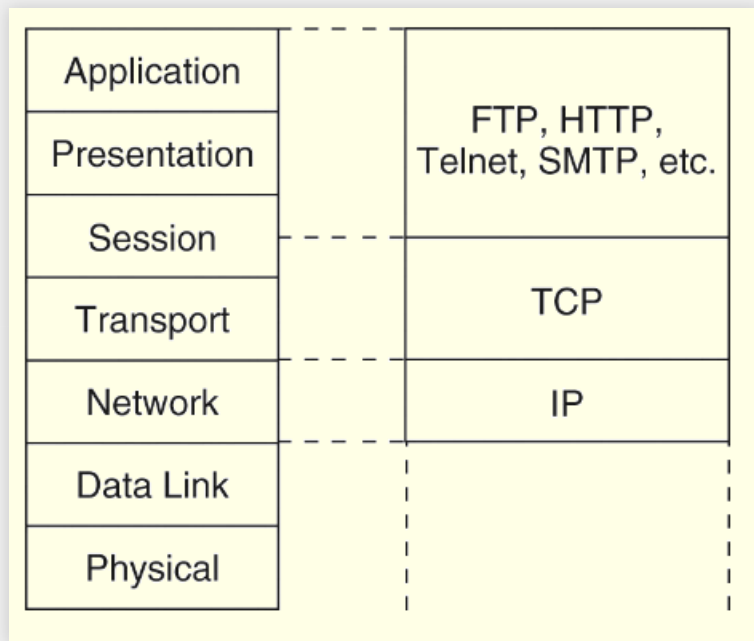
本章总结

- 什么是开门狗定时器？它是如何工作的？
- 什么是PAL, PLA和FPGA?它们是如何实现可编程的特性??
- 嵌入式系统的内存分布是什么样的？
- 什么是中断嵌套？
- 什么是优先级反转？
- 中断嵌套和优先级反转的工作原理是什么？

网络体系结构

TCP/IP 体系结构

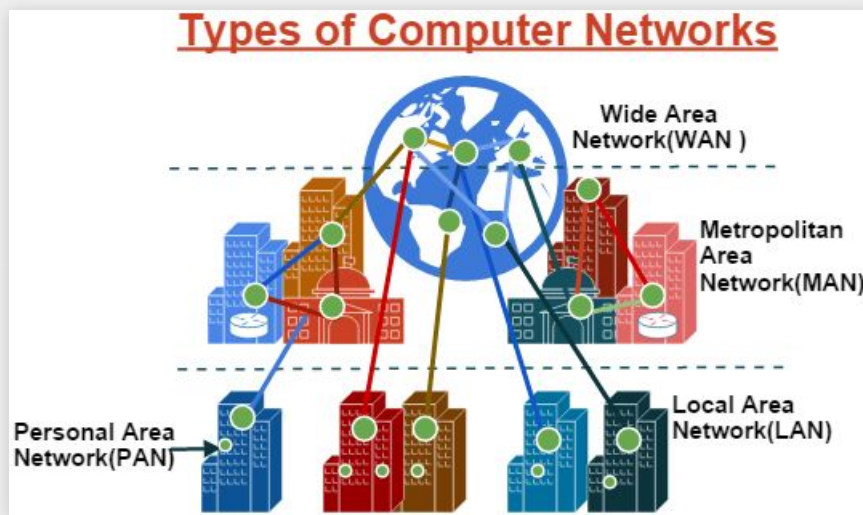
- ISO/OSI 参考模型并未在任何商用网络中真正实现
- TCP/IP是目前事实上的互联网标准体系结构



网络部署类型

根据网络部署的范围和规模，分为下面几类

- 局域网 (Local Area Network, LAN)
- 城域网 (Metropolitan Area Network, MAN)
- 广域网 (Wide Area Network, WAN)



本章总结

- ISO/OSI参考模型的各层分别是什么？
- 如何将TCP/IP体系结构映射到ISO/OSI参考模型？
- LAN、MAN、WAN之间的区别是什么？



Q & A